QuantumPrice

A High-Frequency Algorithmic Pricing & Arbitrage Engine

Prepared By:

Arjun Rathore

2024A4PS0691H

Institution:

BITS Pilani, Hyderabad Campus

Hyderabad, Telangana

Date: July 17, 2026

Contents

1 Introduction

In the modern digital economy, enterprise e-commerce platforms and financial institutions must process millions of market fluctuations daily. When a competitor drops a price or a stock dips, a platform must recalculate its own algorithmic response and execute an update in milliseconds. Traditional CRUD (Create, Read, Update, Delete) architectures backed by standard SQL databases are fundamentally incapable of handling this velocity without inducing severe latency bottlenecks.

The "QuantumPrice" project was engineered to simulate a high-frequency algorithmic arbitrage environment. It is a distributed, multi-tiered microservices architecture that ingests a relentless firehose of simulated competitor data. The system mathematically guarantees profit boundary protection while dynamically undercutting competitors in microseconds. This report documents the Object-Oriented design patterns, multithreaded concurrency controls, and real-time data pipelines that power the engine.

2 System Architecture Overview

The primary objective during the initial design phase was to identify the inherent bottlenecks in standard data processing. A naive approach to pricing updates involves querying an array of products, executing complex `if/else` logic for each item, and synchronously waiting for the update to complete. At a scale of 50,000+ SKUs and thousands of events per second, an $O(N)$ time complexity search completely freezes the execution thread.

To achieve microsecond-level updates while keeping the application web-accessible, the QuantumPrice engine implements a highly decoupled topology divided into three primary tiers:

- **The Fast Path (C++ Core Engine):** An isolated, in-memory processing daemon that utilizes an $O(1)$ Hash Map (`std::unordered_map`) and thread pools to evaluate competitor strategies and compute profit boundaries.
- **The State & Sync Layer (Python + Redis):** An API Gateway that catches UDP packets from the C++ engine and writes them to an ephemeral Redis database, acting as a global state cache.
- **The Presentation Layer (React):** A high-density dashboard that ingests live data points through WebSockets to render competitive price movements in real-time.

3 The C++ Core Engine: Object-Oriented Modeling

3.1 Data Encapsulation and Profit Protection

To prevent fatal algorithmic glitches (e.g., selling items at a massive loss due to extreme market volatility), strict Object-Oriented Encapsulation was enforced. The `Product` class acts as a cryptographic vault. Variables such as `cost` and `price` are strictly private. The `setPrice` method acts as a mathematical guard, verifying that no calculated price ever drops below the wholesale floor.

To ensure thread safety during multi-core execution, each product embeds its own `std::mutex`. This fine-grained locking mechanism allows multiple threads to update different products simultaneously without locking the entire database.

```
class Product {
private:
    string sku;
    double cost;
    double price;
    int stock;
    PricingStrategy *strategy;
    mutex pLock; // Fine-grained mutex for this specific item

public:
    void setPrice(double newPrice, double compPrice) {
        lock_guard<mutex> lock(pLock);
        double oldPrice = price;

        // Mathematical Guardrail: Prevent pricing below wholesale cost
        if (newPrice < cost) {
            price = cost;
        } else {
            price = newPrice;
        }

        // Fire UDP packet to Python Gateway in microseconds
        sendToPython(sku, price, oldPrice, compPrice, cost);
    }
    // ... Accessors omitted for brevity
};
```

Listing 1: The Encapsulated Product Vault (PricingEngine.cpp)

3.2 The Strategy Design Pattern (Polymorphism)

To satisfy the Open-Closed Principle, the system utilizes the **Strategy Design Pattern**. Hardcoding complex `if/else` logic for thousands of products is unmaintainable. Instead, the system relies on an abstract `PricingStrategy` interface. This allows polymorphic

pricing algorithms to be attached to different products and hot-swapped dynamically at runtime without restarting the engine.

```
class PricingStrategy {
public:
    virtual ~PricingStrategy() = default;
    virtual double calcPrice(const Product &prod, double compPrice) = 0;
};

class SafeStrategy : public PricingStrategy {
public:
    double calcPrice(const Product &prod, double compPrice) override {
        double minPrice = prod.getCost() * 1.20; // Enforce strict 20%
   margin
        if (compPrice < minPrice)
            return minPrice; // Deflect competitor if it ruins margins
        return compPrice - 0.01;
    }
};
```

Listing 2: Polymorphic Pricing Logic

4 Concurrency and Inter-Process Communication

4.1 The Producer-Consumer Thread Pool

Processing thousands of events concurrently required a thread-safe **Producer-Consumer Queue**. Spawning a new thread for every incoming competitor event (e.g., 50,000 per second) would cause OS-level context switching overhead, crashing the host.

Instead, a single main thread (The Producer) ingests the data firehose, pushing events into a generic `std::queue`. A fixed pool of worker threads (The Consumers) constantly monitors this queue. They sleep to preserve CPU cycles and awaken via a `std::condition_variable` to process events simultaneously.

```
vector<thread> workers;
int numWorkers = 4; // Capped to prevent Thread Exhaustion

for (int i = 0; i < numWorkers; i++) {
    workers.push_back(thread([&registry]() {
        while (true) {
            pair<string, double> task;
            {
                unique_lock<mutex> lock(qLock);
                // Wait until notified by the Producer
                cv.wait(lock, [] { return !workQueue.empty() || isDone;
   });

                if (workQueue.empty() && isDone) return;

```

```
                task = workQueue.front();
                workQueue.pop();
            }
            // Process event via O(1) Hash Map lookup
            registry.processEvent(task.first, task.second);
        }
    }));
}
```

Listing 3: Producer-Consumer Thread Allocation

4.2 Reverse UDP Bridge: Dynamic Memory Hot-Swapping

The C++ engine must remain online 24/7. To allow administrators to dynamically change a product's strategy from the web dashboard, a two-way Inter-Process Communication (IPC) bridge was constructed. The C++ engine spawns a background daemon thread that binds to UDP port 5001. When it catches a packet from the Python FastAPI server, it safely locks the Hash Map and hot-swaps the physical memory pointer of the object in real-time.

```
void apiListenerThread(SKURegistry *registry, PricingStrategy *safeStrat
    , PricingStrategy *aggStrat) {
    // ... Socket binding initialization omitted ...
    char buffer[1024];
    while (true) {
        int bytes = recvfrom(recvSocket, buffer, 1024, 0, NULL, NULL);
        if (bytes > 0) {
            buffer[bytes] = '\0';
            string msg(buffer);
            vector<string> parts = parseCSV(msg);

            // Catch admin command and execute runtime hot-swap
            if (parts[0] == "STRATEGY" && parts.size() == 3) {
                PricingStrategy *newStrat = (parts[2] == "Safe") ?
    safeStrat : aggStrat;
                registry->updateStrategy(parts[1], newStrat, parts[2]);
            }
        }
    }
}
```

Listing 4: Background API Listener Thread

5 The Gateway and Edge Presentation Layer

5.1 Redis Pub/Sub and FastAPI Integration

The C++ engine is intentionally isolated from HTTP overhead. When a price is calculated, C++ fires a "fire-and-forget" UDP packet to Python. Python utilizes **Redis** as

an ephemeral whiteboard. To bypass standard database polling, Python utilizes the Redis Pub/Sub feature to immediately broadcast the payload to a `price_updates` channel, enabling event-driven architecture.

```python
def udp_listener():
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((UDP_IP, UDP_LISTEN_PORT))

    while True:
        data, _ = sock.recvfrom(1024)
        message = data.decode('utf-8')
        try:
            parts = message.split(',')
            if len(parts) >= 5:
                sku = parts[0]
                # Save to state and immediately shout over Pub/Sub
                r.set(sku, message)
                r.publish("price_updates", message)
        except Exception:
            pass
```

Listing 5: Python UDP Listener and Redis Broadcasting (Bridge_Phase5.py)

5.2 High-Frequency WebSocket Broadcaster

Standard HTTP requests are insufficient for a Level-2 financial trading terminal. Connected React clients maintain an open **WebSocket** pipe to the Python Gateway. The moment Redis publishes a new string, Python catches it asynchronously and pipes it directly into the browser DOM in milliseconds.

```python
@app.websocket("/ws/stream")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    pubsub = r.pubsub()
    pubsub.subscribe("price_updates")

    try:
        while True:
            # Catch Redis shouts without blocking the thread
            message = pubsub.get_message(ignore_subscribe_messages=True)
            if message:
                await websocket.send_text(message['data'])
            await asyncio.sleep(0.01)
    finally:
        pubsub.unsubscribe("price_updates")
```

Listing 6: Asynchronous WebSocket Streaming

5.3 React Presentation and Visual Latency Mitigation

To handle massive data influxes without freezing the user's browser, the React frontend leverages `useEffect` and `useRef` hooks to efficiently batch DOM updates. A dedicated `Row` component tracks its own timestamp. When an algorithmic undercut occurs, the row compares the old price against the new price and triggers a 800-millisecond CSS animation, flashing the row green (undercut success) or red (margin failsafe triggered).

```
function Row({ sku, price, oldPrice, compPrice, cost, timestamp }) {
    const [flashClass, setFlashClass] = useState('');
    const prevTime = useRef(timestamp);

    useEffect(() => {
        if (timestamp !== prevTime.current) {
            // Determine algorithm outcome for visual feedback
            const isPriceDrop = parseFloat(price) < parseFloat(oldPrice)
   ;
            setFlashClass(isPriceDrop ? 'flash-green' : 'flash-red');

            const timer = setTimeout(() => setFlashClass(''), 800);
            prevTime.current = timestamp;
            return () => clearTimeout(timer);
        }
    }, [timestamp, price, oldPrice]);
    // ... Render logic omitted ...
}
```

Listing 7: React Visual Indicator Logic (index.html)

6 High-Resolution Telemetry

In high-frequency systems, standard millisecond timers rely on the operating system clock, inducing inaccuracies. To properly benchmark the architecture, the C++ `std::chrono::high_resoluti` was implemented. This interfaces directly with the hardware's internal crystal oscillator to track the exact microsecond duration elapsed from the ingestion of the first competitor event to the closure of the final worker thread.

```
int eventCount = 0;
auto start = chrono::high_resolution_clock::now();

// ... Event Ingestion Loop ...

// Barrier synchronization: Wait for all queue tasks to finish
for (auto &w : workers) {
    if (w.joinable()) w.join();
}

auto end = chrono::high_resolution_clock::now();
```

```
auto duration = chrono::duration_cast<chrono::microseconds>(end - start)
    .count();

{
    lock_guard<mutex> cLock(consoleLock);
    cout << "[BENCHMARK] Processed " << eventCount << " events in "
         << duration << " microseconds.\n";
}
```

Listing 8: Monotonic Hardware Telemetry (PricingEngine.cpp)

7 Critical Concurrency Resolutions

Building a multi-threaded data pipeline introduced severe edge cases that required strict programmatic solutions.

7.1 Iterator Invalidation (Hash Map Crashing)

When testing the dynamic REST API, instructing C++ to insert a new product (`ADD`) into physical memory altered the size and shape of the Hash Map. Simultaneously, worker threads attempting to query that same Hash Map encountered fatal iterator invalidation, causing Segmentation Faults. **Resolution:** A global `mapLock` (`std::mutex`) was introduced to the `SKURegistry`. While worker threads hold the lock for a fraction of a microsecond during queries, it mathematically guarantees the physical memory matrix cannot change shapes during read operations.

7.2 Cross-Thread Console Smashing

During peak multi-core processing, different threads finished calculations at the exact same microsecond and attempted to write output to the terminal simultaneously, resulting in illegible, interleaved text strings. **Resolution:** The standard output stream (`std::cout`) was identified as a shared resource. A secondary `consoleLock` was implemented, forcing threads to queue for terminal access without halting their background computational tasks.

8 System Execution & Deployment

To rigorously validate the architecture, the system relies on a local Redis instance and compiles the multithreaded C++ engine natively. A custom Python data generator is used to simulate a high-frequency market environment, and an OS-level subprocess manager handles autonomous execution.

Execution Sequence:

1. **Initialize the Global State Cache:** Start the local Redis server on default port 6379 (via Docker or a native Windows installation).

2. **Generate Market Data Firehose:** Execute the data generator script to create the product matrix and 2,000 randomized competitor events.

```
python script.py
```

3. **Compile the C++ Core Engine:** Compile the multithreaded backend, ensuring the threading and networking libraries are linked.

```
g++ -std=c++17 -pthread PricingEngine_Phase11.cpp -o engine.exe -
    lws2_32
```

4. **Boot the API Gateway:** Launch the Python FastAPI server to establish the UDP listener and WebSocket pipelines.

```
python Bridge_Phase5.py
```

5. **Launch the Presentation Dashboard:** Open `index.html` in a modern web browser to connect to the active WebSocket stream.

6. **Ignite the Engine:** Trigger the system via the UI dashboard (which calls the `POST /engine/start` endpoint) to autonomously boot the C++ daemon and process the 2,000 event firehose in real-time.

9 Future Expansion

While the current architecture handles raw computational velocity flawlessly on a single node, its UDP dependency limits horizontal scaling across disparate geographic clusters. A future expansion would replace the local UDP bridge with an enterprise message broker such as **Apache Kafka**. This would permit multiple instances of the C++ Core Engine to scale horizontally across Kubernetes clusters, consuming Kafka topics to achieve high-availability fault tolerance.

10 Conclusion

QuantumPrice successfully demonstrates the execution of enterprise-grade backend architecture. Subjected to a simulated attack of 2,000 high-frequency competitor events across 50 dynamic assets, the system processed the payload entirely in memory in sub-millisecond durations. By harmonizing C++ memory safety with asynchronous Python message brokering and React WebSockets, the architecture guarantees absolute mathematical integrity while delivering a visually responsive, real-time Level 2 trading terminal experience.